

CSI CYBER

CONCEITOS DE MALWARE



3

LISTA DE FIGURAS

Figura 1 – Relação Tempo x Complexidade da análise	12
Figura 2 – Camadas da arquitetura de computadores	13
Figura 3 – Sistemas operacionais por versão	16
Figura 4 – Arquitetura de Neumann	16
Figura 5 – Referenciando dados no <i>registrador</i> EAX	19
Figura 6 – Níveis de abstração.....	21

EMANIP

LISTA DE QUADROS

Quadro 1 – Descrição e uso dos registradores	18
Quadro 2 – Instruções Assembly	22

EMULAP

SUMÁRIO

1 CONCEITOS DE MALWARE	5
1.1 Tipos de malware	5
2 ANTI ENGENHARIA REVERSA	7
3 POLIMORFISMO	8
4 METAMORFISMO.....	9
5 OFUSCAÇÃO E COMPACTAÇÃO	10
6 ANÁLISE DE MALWARE	10
6.1 Engenharia de software.....	12
6.2 Engenharia reversa	14
6.3 Arquitetura Intel x86	15
6.4 Arquitetura de Von Neumann	16
7 REGISTRADORES	18
7.1 EFLAGS	19
7.2 EIP	20
7.3 OPCODES	20
CONCLUSÃO.....	24
REFERÊNCIAS.....	25

1 CONCEITOS DE MALWARE

Apesar dos criadores de malware utilizarem sempre técnicas inovadoras como anti engenharia reversa, ofuscação e compactação a fim de blindarem seus artefatos e, assim, obstruir a análise de seu malware, é por meio da engenharia reversa que os analistas conseguem, em parte, reverter os artefatos para nível de código com ajuda de softwares especializados e, com isto, revelar a finalidade do malware bem como criar assinaturas para IDS (*Intrusion Detection System*), IPS (*Intrusion Protection System*) e Antivírus.

Serão abordados os tipos de análises de malware que o analista tem em mãos para alcançar seu objetivo e descobrir o propósito de cada malware.

Abordaremos também a arquitetura x86, seus registradores e *flags*, bem como as instruções a nível de máquina “opcodes”.

1.1 Tipos de malware

Antes de detalhar e iniciar o processo de análise de malware, precisamos definir alguns conceitos e terminologias, conhecer o processo básico de criação de um software, algumas linguagens de programação e definir alguns tipos de malware.

É de grande importância ter uma sólida base teórica antes de começarmos a analisar um malware. Vários conceitos são de extrema importância para nos guiar neste processo.

Um malware é todo e qualquer software que cause danos ao usuário, computador ou rede, exemplo: Vírus, Cavalo de Troia, Worms, Spyware, Ransomware entre outros. Como podemos observar, o malware poderá se apresentar de várias formas, porém são utilizadas técnicas comuns a todas elas.

É de suma importância desvendar o que o malware está tentando fazer, antes mesmo de começar a análise propriamente dita e, em seguida, confirmar suas hipóteses após sua análise. Claro que você poderá ter mais sucesso em suas “adivinhações” se souber o que faz cada tipo de malware, para isso detalharemos algumas categorias em que se enquadram a grande parte dos malwares.

Worm ou vírus: Código malicioso que tem a capacidade de se reproduzir e distribuir cópias de si mesmo para outros computadores, ocultando-se em arquivos para infectar outros sistemas. Outra característica importante do Vírus/Worm é que ele dependerá de alguma ação do usuário para ser ativado e infectar o computador.

Backdoor: Código malicioso que possibilita acesso remoto a um invasor, permitindo que este execute comandos remotamente em um computador infectado, sem que, para isso, tenha um mínimo de autenticação. O invasor utiliza-se de alguma vulnerabilidade do SO ou de alguma aplicação que esteja executando no computador da vítima.

Botnet: Com características semelhantes ao backdoor, permite ao invasor conectar-se ao sistema infectado. Após a conexão estabelecida, a vítima fará parte de uma rede conectada a um servidor central de comando e controle sob o domínio do atacante e aguardará instruções a serem executadas. Geralmente uma Botnet é utilizada para ataques DDOS (*Distributed denial-of-service*).

Downloader: O Downloader é um código malicioso que baixa outros códigos maliciosos que são instalados por invasores após ganharem acesso ao sistema pela primeira vez.

Data Stealing Malware: Malware utilizado para coletar informações pessoais como: logins, e-mails e senhas das vítimas e enviá-las ao invasor. De posse destes dados, o invasor poderá obter acesso às contas dos serviços on-line e serviços bancários da vítima. Fazem parte desta categoria os sniffers, keyloggers, screenloggers entre outros.

Rootkit: Código malicioso desenvolvido para ocultar outro código, são combinados com outros malwares como keyloggers ou backdoors. Geralmente são injetados no kernel do SO ou em firmwares. Detectar um rootkit não é muito trivial. Há casos, quando estes residem no kernel, em que apenas a reinstalação do sistema operacional removerá o rootkit.

Trojan: Código malicioso entregue pelo atacante, à vítima, na forma de software legítimo (download-de-vídeos.exe, serial.exe, antivírus.exe, jogo-original.exe) que trará a solução para alguns dos problemas da vítima. Porém, vem carregado com software malicioso que abrirá a porta para o atacante. Seu nome é derivado de uma antiga história grega, onde um cavalo de madeira foi entregue à

cidade de Troia pelos gregos. Dentro deste cavalo havia vários soldados gregos, que esperaram a noite chegar para invadir a cidade de Troia. Famoso “presente de grego”.

Um malware geralmente pode abranger várias categorias, por exemplo: um trojan pode ter um keylogger para coletar senhas e um worm que envia spam. Não devemos nos prender em classificar malware pelas suas categorias, e sim conhecê-las para ajudar a identificá-los rapidamente.

2 ANTI ENGENHARIA REVERSA

Frequentemente, os autores de malwares precisam blindar seu malware para que eles sejam imunes a engenharia reversa, pois o autor do malware não deseja que seu código seja descoberto facilmente por outros hackers ou analistas de malware e não seja descoberto, também, por meio de antivírus, IDS ou IPS.

Não é possível impedir totalmente a reversão. O que é possível é impedir e obstruir os inversores, desgastando-os e tornando o processo tão lento e doloroso que eles simplesmente desistem. Se alguns reversores terão sucesso, depende de vários fatores, como a capacidade deles e a sua motivação (tradução do autor) (Eilam, 2005).

Existem diversas abordagens para proteger o código malicioso, podendo o autor do malware utilizar-se mais de uma. Abaixo iremos abordar algumas mais conhecidas.

- **Anti-Disassembly:** As técnicas de anti-disassembly são utilizadas para atrasar ou até impedir a análise do código por meio de ferramentas de disassembly. Autores de malware usam esta técnica para que seja exigido um nível maior de habilidade por parte do analista de malware.
- **Anti-Debugging:** Nesta técnica, o malware detecta que está sob controle de um depurador e pode alterar seu fluxo de execução ou até modificar o código para causar uma quebra na execução. O autor do malware pode utilizar a própria API do Windows, verificação manual da estrutura de memória para depuração de artefatos ou busca resíduos deixados por um depurador na memória.

- **Anti-VM:** Nesta técnica, o autor do malware tenta impedir que ele seja analisado em ambientes virtuais, ou seja, dentro de uma máquina virtual. Quando o malware detecta que está sendo executado numa VM, ele simplesmente para a sua execução ou funciona de forma incorreta, causando problemas para o analista de malware.
- **Packers:** Uma das técnicas mais populares entre os desenvolvedores de malware é a ofuscação do malware. Com esta técnica é possível esconder o malware dos softwares de antivírus, dificultar a análise de código e reduzir o tamanho do executável. Há na internet, disponível gratuitamente, packers bastante fáceis de usar. Grande parte dos packers usam algoritmo de compactação e criptografia no executável original (malware) criando outro executável.

3 POLIMORFISMO

Uma das maneiras mais fáceis para software de antivírus identificarem programas maliciosos é por meio de assinaturas exclusivas. Os melhores softwares de antivírus guardam uma base de dados de assinaturas de softwares maliciosos onde são mantidas diversas assinaturas de malwares, cada uma relacionada a um malware específico. Essas assinaturas são frequentemente atualizadas sempre que um novo malware é identificado.

A assinatura de um malware é uma sequência de bits única encontrada em uma amostra do malware conhecido.

O polimorfismo, em malwares, é uma técnica que impedirá que essas assinaturas sejam facilmente identificadas por antivírus, pois, com o polimorfismo, é possível criptografar ou codificar aleatoriamente o código em tempo de execução para que, quando executado, retorne a forma original do malware para infectar seu alvo.

O polimorfismo baseado em criptografia com chave aleatória é a forma mais simples do uso em malware, já que é por meio da descryptografia, em tempo de execução, que o malware habitará a memória do sistema. Com isso os softwares de antivírus não poderão criar uma assinatura única para este tipo de malware, uma

vez que cada cópia será totalmente diferente da outra, dependendo da chave aleatória que o malware utilizará para a sua descriptografia.

Contra este tipo de malwares, os softwares de antivírus podem procurar por assinaturas de malware diretamente na memória. Na maioria dos casos, o software malicioso estará presente na memória do sistema após ser descriptografado, então o antivírus criará uma assinatura entre o malware carregado na RAM e sua cópia no disco.

Para os antivírus que procuram por softwares maliciosos diretamente no disco, pode-se utilizar o próprio código de descriptografia como assinatura. Mesmo a chave sendo aleatória, o código de descriptografia será estático.

4 METAMORFISMO

Como podemos observar em códigos polimórficos, as modificações superficiais no código de descriptografia permitirão que o malware seja identificado por antivírus. É aqui que entra o metamorfismo.

Diferentemente dos códigos polimórficos, cuja alteração é feita no corpo do malware e pequenas alterações no mecanismo de criptografia, o metamorfismo é capaz de alterar todo o malware (mecanismo de criptografia e corpo do malware) para cada cópia replicada. Esta técnica visa livrar o malware de assinaturas de antivírus, ou seja, cada versão do malware replicada será totalmente diferente da outra, isso torna impossível a utilização do antivírus para identificação do malware.

Códigos metamórficos contam com mecanismo de análise de código poderoso que analisa todo o programa e regenera diferentes versões em tempo real.

Mecanismos metamórficos têm a capacidade de:

- Inserir aleatoriamente instruções de lixo que manipulam dados sem relevância causando confusão aos analistas de malware.
- Reordenar funções do software original.
- Alterar, de forma aleatória, instruções dentro de uma função.

- Inverter condicionais em que, por exemplo, uma instrução que verifica se A é igual a B é transformada em A é diferente de B.

5 OFUSCAÇÃO E COMPACTAÇÃO

Para dificultar a análise estática de um malware, os criadores de códigos maliciosos usam a ofuscação para que o programa malicioso não possa ser analisado. Malwares ofuscados são criados para serem ocultados até sua execução. A compactação é um subconjunto da ofuscação onde softwares maliciosos são compactados para não serem analisados, limitando intensamente as tentativas de análise estática.

6 ANÁLISE DE MALWARE

É por meio da análise de malware que serão geradas informações necessárias para identificar o tipo de ameaça, quais suas capacidades e quão crítico esse ataque pode ser. Com base nessas respostas, poderemos responder a uma invasão de rede ou host. Após levantadas todas informações, serão geradas assinaturas baseadas em rede e/ou host para proteção deles.

Frequentemente, ao realizar uma análise de malware, teremos apenas um arquivo binário executável que são um conjunto de 0s e 1s sem sentido algum para nós humanos. Para que possamos entender e dar sentido ao executável poderemos utilizar uma vasta variedade de ferramentas para levantar, aos poucos, informações sobre o malware.

A análise de malware pode ser dividida em duas categorias: a estática e a dinâmica, sendo que cada uma delas pode se dividir em básica e avançada.

- **Análise Estática Básica:** Baseia-se na verificação do arquivo binário/executável, sem ter que analisá-lo em relação ao código. Basicamente tentaremos extrair o máximo de informação como: hash, strings, se o malware está empacotado, ou seja, passou por um packer, confirmar se o arquivo é realmente um malware. Informações bastante simples, porém, caso o malware seja muito sofisticado, não teremos muito

progresso nesta fase. Podemos também utilizar o site [<https://www.virustotal.com/>](https://www.virustotal.com/) para fazer uma análise estática básica inicial.

- **Análise Estática Avançada:** Consiste na engenharia reversa do malware. Aqui é necessário rodar o malware em ferramentas de disassembly para estudar todo código e lógica computacional enquanto suas instruções são executadas em CPU e armazenadas na memória RAM. Este tipo de análise requer do analista um amplo conhecimento em disassembly, chamadas de sistema ao núcleo do sistema operacional. Com as informações obtidas nesta análise, teremos a capacidade de gerar assinaturas para hosts e redes.
- **Análise Dinâmica Básica:** Esta técnica consiste em analisar o comportamento do malware no sistema após sua execução e observar seu comportamento no sistema. É de extrema importância que a VM, onde o malware será executado, esteja totalmente isolada para proteger seu host e sua rede. Para uma análise segura, podemos utilizar alguns sites que podem fazer essas análises de forma totalmente automatizada e sem risco de infectar nossa infraestrutura, por exemplo: [<https://any.run/>](https://any.run/) e [<https://hybrid-analysis.com/>](https://hybrid-analysis.com/).
- **Análise Dinâmica Avançada:** Esta técnica é bastante útil quando não conseguimos reunir informações com as outras técnicas. A utilização de *debuggers* nesta técnica é essencial para que seja possível analisar o estado interno do malware.

Cada tipo de análise tem seu nível de complexidade e quantidade de tempo despendido para conclusão. Como podemos ver na figura “Relação Tempo x Complexidade da análise”, a análise dinâmica básica requer pouco conhecimento e o tempo despendido também é bem menor se comparado com a análise estática avançada, por exemplo.

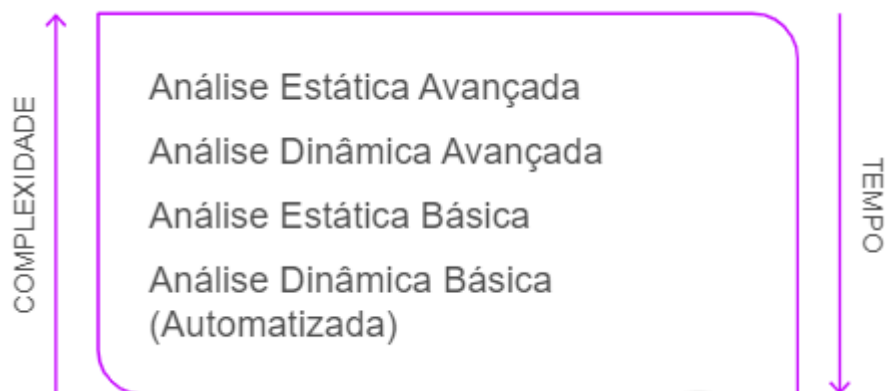


Figura 1 – Relação Tempo x Complexidade da análise
Fonte: Elaborado pelo autor (2020)

6.1 Engenharia de software

Para um computador ter a capacidade de resolver uma enorme série de problemas, que podem ser um simples emulador de jogos dos anos 80 a virtualizar sistemas operacionais inteiros dentro de outro sistema operacional, precisa executar programas escritos em diversas linguagens.

A experiência de programação nos ensina que quando um problema é grande, devemos quebrá-lo e usar uma abordagem de dividir para conquistar. Na programação, dividimos um problema em módulos e, em seguida, projetamos cada módulo separadamente. Cada módulo executa uma tarefa específica, e os módulos só precisam saber como interagir com outros módulos para fazer uso deles (NUL; LOBUR, 2014, tradução do autor).

É através da abstração que podemos dividir a arquitetura de computadores em diferentes níveis e, assim, criar camadas que executam apenas seus conjuntos de instruções e se interligam umas com as outras, como podemos ver na figura “Camadas da arquitetura de computadores”.

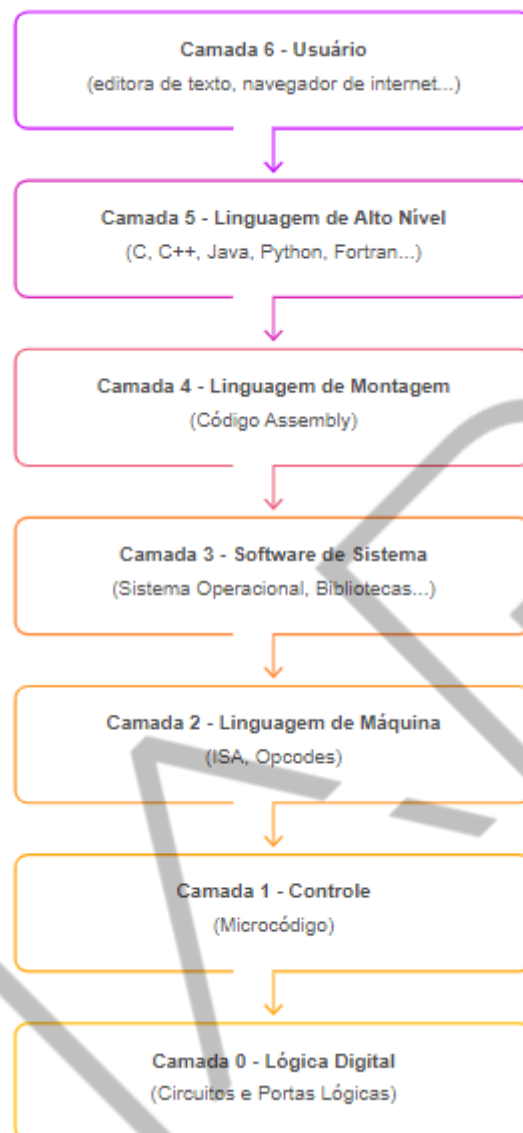


Figura 2 – Camadas da arquitetura de computadores
Fonte: Elaborado pelo autor (2020)

- **Camada 6:** Camada do usuário onde se encontram os aplicativos com os quais todos estão acostumados a trabalhar, por exemplo: navegadores de internet, jogos, editores de texto, player de músicas etc.
- **Camada 5:** Camada das linguagens de Alto Nível como C, C++, Fortran, Pascal, Python etc. Após ser traduzida, por meio de um compilador ou interpretador, essas se transformam em uma linguagem de máquina, ou seja, o que a máquina realmente vai executar.
- **Camada 4:** Camada de linguagem de montagem ou de baixo nível. Nesta camada, a linguagem de alto nível é primeiramente transformada em

linguagem de montagem e depois traduzida diretamente para linguagem de máquina transformando-se em 0s e 1s.

- **Camada 3:** Camada do Sistema Operacional. Esta camada trata as instruções do sistema operacional e é responsável por multiprogramação, proteção de memória, sincronização de processos etc.
- **Camada 2:** Camada da Arquitetura de Conjunto de Instruções ou ISA (*Instruction Set Architecture*) ou nível de máquina, linguagem reconhecida pela arquitetura do sistema computacional. Programas escritos em linguagem de máquina são executados diretamente em circuitos eletrônicos sem a necessidade de qualquer interpretador, tradutor ou compilador.
- **Camada 1:** Camada de Controle. É onde a unidade de controle certifica que as instruções sejam adequadamente decodificadas e executadas bem como devem ser movidos seus dados. Todas as instruções nesta camada são executadas uma de cada vez.
- **Camada 0:** Camada de Lógica Digital. São os componentes físicos do sistema computacional e não são facilmente modificados.

6.2 Engenharia reversa

É a extração de conhecimento ou informações de qualquer coisa feita pelo homem e a reprodução de tais coisas com base nas informações extraídas. Muitas vezes a engenharia reversa é utilizada para descobrir projetos de concorrentes na indústria.

Como estamos falando de um malware (*Malicious Software*), a engenharia reversa que utilizaremos é a de software. Para este fim, o analista de malware necessita de uma combinação de habilidades e ótima compreensão de desenvolvimento de software. A engenharia reversa de software incorpora algumas artes, como quebra de código e resolução de quebra cabeça e o pré-requisito mais importante é a vontade e a curiosidade de aprendizado.

Em análise de malware, é utilizado um software de *disassembly*, que tem como entrada um arquivo binário e gera um código em linguagem de baixo nível *assembly*, para que seja possível o analista de malware entender a lógica por trás do malware e como ele foi construído.

Como vimos anteriormente, em camadas de software, a linguagem *assembly* está na camada de baixo nível, ela é que separa os humanos e softwares do hardware propriamente dito.

Anos atrás, os programadores sempre tiveram que trabalhar neste nível baixo porque era a única maneira possível de escrever um software. Hoje em dia, os modernos sistemas operacionais e ferramentas de desenvolvimento buscam nos isolar dos detalhes do mundo de baixo nível. Isso simplifica muito o processo de desenvolvimento de software, mas acarreta redução de potência e controle do sistema. A linguagem *assembly* é o nível mais baixo na cadeia de software, o que a torna incrivelmente adequada para reversão, nada se move sem ela. Se o software executa uma operação, ela deve estar visível no código da linguagem *assembly* (ELIAM, 2005, tradução do autor).

6.3 Arquitetura Intel x86

A arquitetura Intel x86 (IA-32) (arquitetura de 32 bits da Intel) é a base para todos os processadores x86, desde os primeiros processadores 80386 até os processadores de hoje. Grande parte dos computadores pessoais de 32 bits é x86. Outrossim, as arquiteturas AMD64 e IA-64, de 64 bits, executam Windows que oferecem suporte a aplicações de 32 bits. Tendo em vista essa compatibilidade é que os desenvolvedores de malwares compilam seus códigos para esta arquitetura. Toda teoria inserida nesta apostila refere-se à arquitetura x68 (IA-32), bem como todos os malwares analisados neste curso também serão nesta arquitetura, caso contrário será explicitamente informado.

Hoje 87% dos desktops e notebooks executam o sistema operacional Windows, por isso o foco dos desenvolvedores de malware é este SO. Se olharmos pelo ponto de vista do desenvolvedor de malware, é mais fácil infectar um alvo com Windows, tendo em vista que a quantidade de computadores executando este sistema operacional é muito superior a computadores executando o sistema operacional Linux.



Figura 3 – Sistemas operacionais por versão
Fonte: Net Market Share (2020)

6.4 Arquitetura de Von Neumann

A arquitetura x86, bem como a maioria das arquiteturas dos computadores atuais, segue a arquitetura de Von Neumann. Esta arquitetura é composta por 3 componentes internos de hardware (CPU, Memória principal e Sistemas de Entrada e Saída).

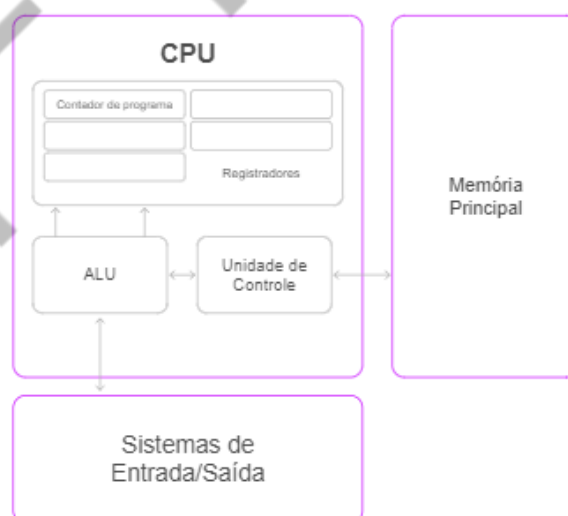


Figura 4 – Arquitetura de Neumann
Fonte: Elaborado pelo autor (2020)

A principal função da CPU é buscar as instruções de programas que estão armazenados na memória principal, decodificá-las e executá-las sequencialmente.

Como podemos ver na figura “Arquitetura de Neumann”, a CPU contém alguns componentes, sendo os principais:

- Unidade de Controle: Busca as instruções na memória principal e coordena a execução das instruções armazenando o endereço da próxima instrução no registrador contador de programa ou *Instruction pointer* (EIP);

O registrador mais importante é o Contador de Programa (PC – Program Counter), que indica a próxima instrução a ser buscada para execução. (O nome “contador de programa” é um tanto enganoso, porque nada tem a ver com contar qualquer coisa; porém, o termo é de uso universal.) Também importante é o Registrador de Instrução (IR – Instruction Register), que mantém a instrução que está sendo executada no momento em questão. A maioria dos computadores também possui diversos outros registradores, alguns de uso geral, outros de uso específico (TANEMBAUM; TODD, 2013).

- A Unidade Lógica e Aritmética (ALU — do inglês, *Arithmetic and Logic Unit*): É onde as operações lógicas e aritméticas são realmente realizadas, aqui reside a essência de um computador, pois todos os outros elementos (unidade de controle, registradores, memória principal e E/S) têm o objetivo de alimentar a ALU para processamento dos dados;
- Registradores: Onde são armazenados os dados que serão utilizados internamente pela CPU. São considerados como unidade básica de altíssima velocidade, poupando tempo ao processador, pois não necessita buscar o dado novamente na memória RAM;
- Contador de Programa: Endereço da próxima instrução a ser buscada na memória.

Cada instrução é executada pela CPU seguindo um ciclo de instrução, divididas em pequenas etapas a saber:

- **1ª Etapa:** Buscar a próxima instrução na memória RAM e armazená-la no Registrador de Instrução.
- **2ª Etapa:** Com a instrução salva, o registrador contador de programa é incrementado e passa a apontar para a próxima instrução.
- **3ª Etapa:** É determinado o tipo da instrução trazida.

- **4ª Etapa:** Caso haja a necessidade de utilização de algum dado na memória RAM, esse dado é trazido para dentro da CPU e armazenado dentro de um registrador.
- **5ª Etapa:** Executa a instrução.
- **6ª Etapa:** Volta para a etapa 1.

7 REGISTRADORES

É o menor local físico, para armazenamento de dados ou endereço de memórias, dentro da CPU e servem para controlar a execução do programa, salvar resultados temporários e outras finalidades.

Os registradores são divididos em 2 categorias: os de uso geral e os de uso específico. Os GPRs (*General Purpose Register*) são utilizados para armazenar temporariamente todo e qualquer tipo de dado e, por convenção, são utilizados da seguinte maneira:

REGISTRADORES	DESCRIÇÃO	USO
EAX	ACCUMULATOR REGISTER	Operações aritméticas
EBX	BASE REGISTER	Ponteiro para dados
ECX	COUNTER REGISTER	Contador em repetições
EDX	DATA REGISTER	Operações de E/S
ESI	SOURCE INDEX	Ponteiro para string de origem
EDI	DESTINATION INDEX	Ponteiro para string de destino
EBP	BASE POINTER	Ponteiro para a base da pilha
ESP	STACK POINTER	Ponteiro para o topo da pilha

Quadro 1 – Descrição e uso dos registradores
Fonte: Elaborado pelo autor (2020)

Dos oito registradores de uso geral, quatro deles (EAX, EBX, ECX e EDX) podem ser referenciados, além dos 32 bits, com 16 ou 8 bits, como podemos ver na figura “Referenciando dados no *registrador* EAX”.

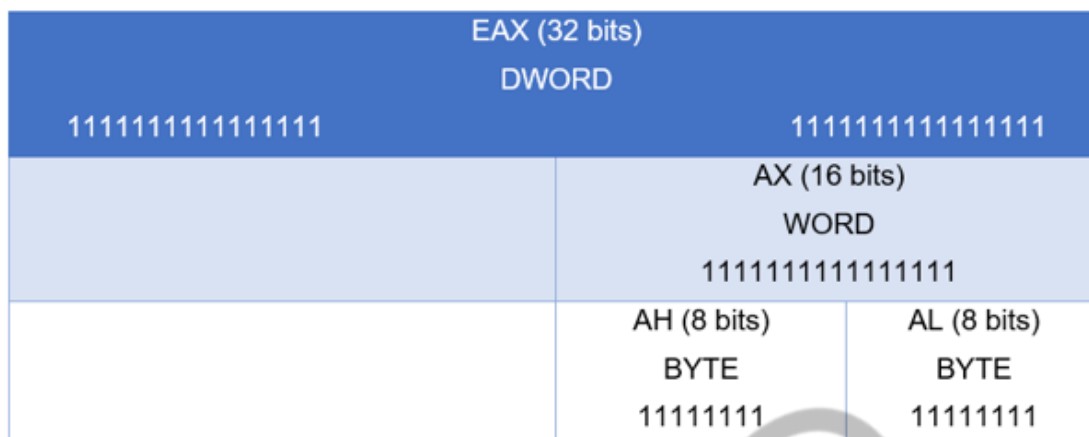


Figura 5 – Referenciando dados no *registrador* EAX
 Fonte: Elaborado pelo autor (2020)

7.1 EFLAGS

Registrador específico de flags que contém todos os estados do sistema. Na arquitetura IA-32 (x86) esse registrador possui 32 bits que podem ser setados com 0 ou 1 para cada flag. Os vários modos e estados do processador são gerenciados pelas flags do sistema e são irrelevantes para esta discussão. Por outro lado, as flags de status são usadas pelo processador para registrar seu estado lógico atual, registrando o resultado de suas ações. As flags são uma ferramenta básica para criar código condicional.

DICA: Uma flag será considerada ligada quando seu valor for verdadeiro, ou seja, igual a 1.

Abaixo estão as flags mais importantes para a análise de malware:

- ZF: A *Zero Flag* é setada (1) quando o resultado de uma operação é igual a zero; para outros resultados ela é desligada (0).
- CF: A *Carry Flag* é setada (1) quando o resultado de uma operação é muito grande ou muito pequeno para um operando, para outros resultados ela é desligada (0).
- SF: A *Sign Flag* é ligada (1) quando o resultado de uma operação é negativo ou desligada (0) quando o resultado é positivo. Essa flag também é ligada quando o bit mais significativo é ligado após uma operação aritmética.

- TF: A *Trap Flag* é usada para *debugging*. O processador x86 executará somente uma instrução por vez quando essa flag estiver ligada (1).

7.2 EIP

Ponteiro de instrução estendido (*Extended Instruction Pointer*), aponta para a próxima instrução que o processador deverá executar, não podendo ser inserido qualquer valor diferente de uma posição de memória. Serve para controlar o fluxo de execução de um software.

A CPU possui um registro especial chamado EIP; ele contém o endereço da próxima instrução a ser executada. Quando a instrução for executada, o EIP estará apontando para a próxima instrução na memória (MONNAPPA, 2018, tradução do autor).

DICA: Buffer overflow é quando conseguimos tomar o controle da aplicação por meio do EIP e executar um código malicioso inserido em memória, pois a CPU não será capaz de determinar se a próxima instrução a ser executada é um código legítimo ou malicioso.

7.3 OPCODES

Como foi abordado no item “Engenharia de software” deste capítulo, vimos que a arquitetura de computadores é dividida em camadas e na camada 2 encontramos os *opcodes*.

Os malwares são criados com códigos em linguagem de alto nível que são compilados, gerando assim, o código de máquina que será executado pela CPU.

Em contrapartida, o analista de malware se utiliza da linguagem de baixo nível usando um desmontador (*disassembler*) que, a partir do código de máquina, gera o código de montagem que poderá ser analisado e descobre o funcionamento do software, no nosso caso o malware.

O código de máquina gerado pelo *disassembler* será representado em sistema de numeração hexadecimal para melhor visualização e entendimento para nós humanos, pois com 2 dígitos de em hexadecimal, podemos representar 8 bits ou 1 byte em binário.

Os *opcodes* são apenas um acrônimo de *operation code* ou código de operação e, como seu próprio nome diz, é o código que o processador deverá executar.

Na figura “Níveis de abstração” temos a visão de cada um dos atores envolvidos: o autor do malware, o processador e o Analista do malware.

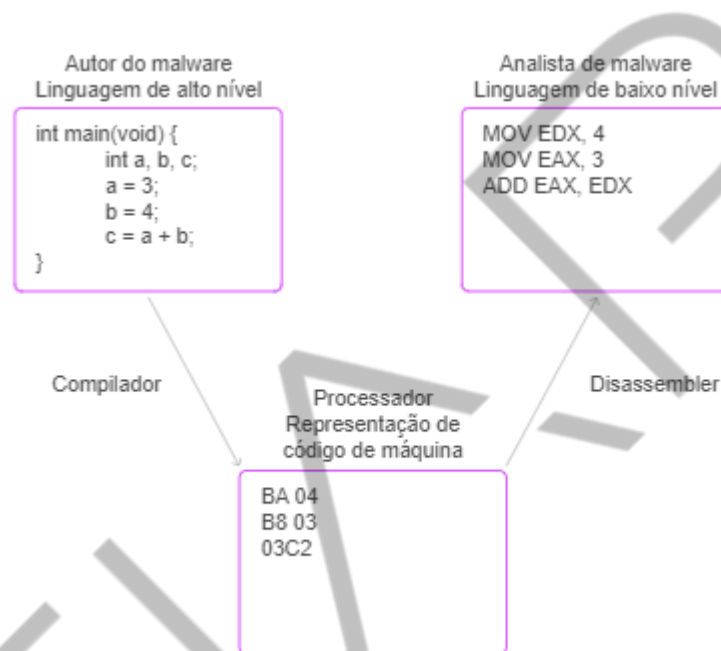


Figura 6 – Níveis de abstração
Fonte: Elaborado pelo autor (2020)

“Ô Hugo, mas eu li em algumas publicações que o código de máquina é em binário, pois o processador só consegue entender 0’s e 1’s, ou seja, ligado ou desligado.”

Ok, caro aluno, você tem toda razão, mas como pode observar na figura “Níveis de abstração”, está escrito “representação de código de máquina”, sendo esta a melhor compreensão para nós, meros mortais. Considerando o código acima em hexadecimal (representação de código de máquina), compare agora o mesmo código em binário e observe qual dos dois você compreende melhor.

10111010 00000100 10111000 00000011 00000011 11000010

Como podemos observar, cada instrução, após o malware compilado, transforma-se em *opcode* que, por sua vez, após traduzida por um *disassembler*, se transforma em instruções de linguagem de baixo nível tornando-se mais legível.

As instruções em linguagem *assembly* são blocos de códigos compostos por um código mnemônico (MOV → *move* → mover) e opera os dados que serão manipulados pela instrução.

No quadro “Instruções Assembly” vemos alguns exemplos de instruções *assembly*.

INSTRUÇÕES				
MNEMÔNICO	OPERANDO DE DESTINO	OPERANDO DE ORIGEM	OPCODE	DESCRIÇÃO
MOV	EDX	4	BA 04000000	Move o valor de 0x4 em hexadecimal para o registrador EAX.
ADD	EAX	EBX	03C3	Soma o valor que está em EDX ao EAX e armazena este resultado em EAX.
NOP	-	-	90	Sem operação (<i>No Operation</i>), ou seja, não executa instruções.
PUSH	EBP	-	55	Insere o valor de EBP dentro da pilha.
POP	EBP	-	5D	Retira o valor de EBP da pilha.
CMP	EAX	0xF149	3D 49F10000	Compara se os dois operandos são iguais. Subtrai de EAX o valor de 0xF149, caso o resultado seja 0, então são iguais, caso negativo EAX é maior que 0xF149, caso positivo EAX é menor que 0xF149. O CMP seta a flag ZF para 1 caso os operandos sejam iguais.
JMP	[EBP-0x08]	-	FF65 F8	Salto incondicional, neste caso faz um salto para o endereço de memória 00444525, ou seja, transfere o controle do programa para um ponto diferente no fluxo de instruções.
INC	EAX	-	40	Incrementa com 1 o operando de destino.
DEC	EAX	-	48	Decrementa em 1 o operando de

Quadro 2 – Instruções Assembly
Fonte: Elaborado pelo autor (2020)

Cada instrução tem seu *opcode* correspondente que informará ao processador a operação que deverá ser realizada.

Tomando como exemplo a instrução `CMP EAX, 0xF149` e seu respectivo *opcode* `3D 49F10000` no quadro “Instruções Assembly” observem que o valor do operando de origem é invertido quando este valor é enviado para a memória, o byte menos significativo é ordenado primeiro dentro da memória parecendo-se com `0x00000040`. Isto é chamado de *endianness*, a ordenação dos dados, o malware ou um software qualquer deve saber lidar com isto, pois na arquitetura x86 é usada a ordenação *little-endian*, no entanto, caso o malware precise se conectar a um IP `127.0.0.1`, este IP deverá estar na memória no formato *little-endian* `0x1000007F` e pela rede seguirá o formato *big-endian* `0x7F000001`.

Os operandos identificam os dados que serão utilizados pela instrução e podem se dividir em três tipos:

- Imediatos: Valores fixos como `0x04`.
- Registradores: O operando refere-se a um registrador, como `EAX`.
- Endereço de memória: O operando refere-se a endereços de memória que contenham o valor a ser utilizado e normalmente são representados por um valor, registrador ou equação, todos entre colchetes: `[0xF1A90345]`, `[ECX]` ou `[EAX+0x16]`.

É de extrema importância que o analista de *malware* esteja familiarizado com as instruções, *opcodes* e operandos. No quadro “Instruções Assembly”, vimos algumas instruções básicas e uma breve descrição. Para mais informações oficiais e uma referência completa, baixe o manual da Intel, que contém a referência completa do conjunto de instruções, em <https://software.intel.com/content/www/us/en/develop/download/intel-64-and-ia-32-architectures-sdm-combined-volumes-2a-2b-2c-and-2d-instruction-set-reference-a-z.html>.

CONCLUSÃO

Concluimos uma grande parte conceitual da análise de malware passando por tipos de malware, tipos de análise de malware e engenharia reversa até chegar na arquitetura Intel x86 onde conhecemos os registradores, *opcodes* e instruções *assembly*. Esses conceitos são de extrema importância para continuarmos nos próximos capítulos.

Grande parte desses conceitos servirão para engenharia reversa de malware. É na análise estática avançada que muitas vezes descobrimos características e funcionalidades do malware que não é possível em uma análise dinâmica.

REFERÊNCIAS

EILAM, E. **Reversing**: Secrets of Reverse Engineering. Wiley, 2005.

MONNAPPA, A. K. **Learning Malware Analysis**. Packt Publishing, 2018.

NULL, L.; LOBUR, J. **Essentials of Computer Organization and Architecture**. 4. ed. Jones & Bartlett Learning, 2014.

TANENBAUM, A. S.; TODD, A. **Organização estruturada de computadores**. 6. ed. Pearson Prentice Hall, 2013.

EMANIP